Part1 - Procedure

1.1 GA Workflow

Each generation consists of five stages: evaluate, select, crossover, mutate and populate. The evaluation stage computes the fitness of all 50,000 chromosomes and sorts them by distance. The best 25,000 chromosomes are selected as parents. The crossover stage produces 25,000 offsprings, which are then mutated and copied into the next generation.

1.2 Parallelization Strategies

First, we shouldn't parallelize the for loop in the main function because each generation is sequential and dependent. We should parallelize five stages for each generation especially the computationally intensive function like evaluate and crossover.

- Evaluate()

The evaluate stage is parallelized with OpenMP. Each loop iteration evaluates one independent chromosome. Because each iteration only reads the chromosome and the distance matrix and writes to its own fitness field, there is no data dependency and synchronization between iterations. Therefore, the chromosomes can be divided among multiple threads without requiring locks or critical section. Although the fitness is parallelized, but the sort is sequential.

```
// Evaluates each trip (or chromosome) and sort them out
void evaluate(Trip trip[CHROMOSOMES], int coordinates[CITIES][2]) {
    #pragma omp parallel for
    for (int i = 0; i < CHROMOSOMES; i++) {
        float totalDistance = 0.0f;
```

- Crossover()

The crossover stage is parallelized with OpenMP because i-th titeration reads parents[i] and parents[i+1] and write only to offspring[i] and offspring[i+1], so each thread is independent and won't affect each other.

```
void crossover( Trip parents[TOP_X], Trip offsprings[TOP_X], int coordinates[CITIES][2] )
    #pragma omp parallel for
    for (int i = 0; i < TOP_X; i += 2) {
```

- Mutate()

The mutate is parallelized with OpenMP because each chromosome's mutation is independently computed. It won't affect each other, so we can create threads to accelerate the process.

```
 * Mutate a pair of genes in each offspring.
 */
void mutate( Trip offsprings[TOP_X] ) {
    #pragma omp parallel for
    for (int i = 0; i < TOP_X; i++) {
```

Part2 - Execution Output

2.1 Results

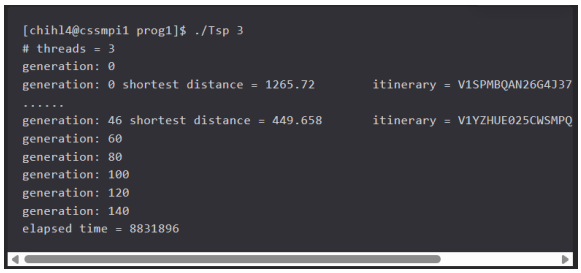
- Thread 1:

```
[chihl4@cssmpi1 prog1]$ ./Tsp 1
# threads = 1
generation: 0
generation: 0 shortest distance = 1265.72  itinerary = V1SPMBQAN26G4J37DX8OTF
......
generation: 85 shortest distance = 449.658 itinerary = V1YZHUE025CWSMPQBD3R7L
generation: 100
generation: 120
generation: 140
elapsed time = 21625921
```

- Thread 2

```
[chihl4@cssmpi1 prog1]$ ./Tsp 2
# threads = 2
generation: 0
generation: 0 shortest distance = 1265.72        itinerary = V1SPMBQAN26G4J37
......
generation: 107 shortest distance = 449.658      itinerary = V1YZHUE025CWSMPQ
generation: 120
generation: 140
elapsed time = 12295133
```

- Thread 3



The shortest trip in my program is equal to or less than 449.658

And the performance improvement with four threads in your program is 3.029 times faster than the thread 1 baseline provided by Professor.

Threads	Run1(s)	Run2(s)	Run3(s)	Average(s)	Speedup
1 (baseline)				23.543059	1.00
1	21.625921	21.609085	21.648727	21.627911	1.088549
2	12.295133	12.309465	12.441525	12.348708	1.906520
3	8.831896	8.881993	8.841677	8.851855	2.659675
4	7.096481	7.126660	7.195926	7.139689	3.297490
4 Compiler optimization	1.178095	1.204193	1.196048	1.192779	19.737989

2.2 Improvement of execution

I originally call the function and repeated calculate the distance between cities. I improve it by precomputing the results and store it in a global 2-dimensional array, then obtain the results with O(1) complexity.

In addition, I replace rand() from stdlib with mt19937 from random library to improve parallelism because of the thread-local and lock-free property.

Besides, I optimize the compiler with O2 Flags to accelerate the execution time by inline function and loop optimization, which improves the execution time a lot for thread 4 from average time 7.13(s) to 1.19(s). It has 19.73 times faster than the baseline.

```
//Global announcement
float distanceMatrix[CITIES+1][CITIES];

// Evaluate() function
for (int gene = 0; gene < CITIES; gene++) {
    int city = cityIndex(trip[i].itinerary[gene]);

    totalDistance += distanceMatrix[previousCity][city];

    previousCity = city;
}
// Crossover() function
else{
    int nextCityDiff1 = distanceMatrix[cityIndex(nextCity1)][previousCityIndex];
    int nextCityDiff2 = distanceMatrix[cityIndex(nextCity2)][previousCityIndex];
    selectedCityIndex = nextCityDiff1 < nextCityDiff2
        ? cityIndex(nextCity1)
        : cityIndex(nextCity2);
}
```

```
mt19937& crossoverRng() {
    thread_local std::mt19937 rng(11111111u + omp_get_thread_num());
    return rng;
}
// Randomly pick up the city in crossover()
mt19937& rng = crossoverRng();
uniform_int_distribution<int> dist(0, unusedCityIndices.size() - 1);
selectedCityIndex = unusedCityIndices[dist(rng)];
```

Part3 – Discussion

The OpenMP implementation improved the execution time because the evaluation and crossover operations contain many independent iterations so that we can easily speed up and avoid synchronization.
But the speedup is not perfectly linear because sorting and some functions remain sequential. The sequential sorting phase limits the overall speedup according to the Amdahl's law.

I use the gprof profiler (Data is in analysis.txt) to analyze the program and it turns out the sequential function followingCity() in the crossover() function is the huge bottleneck occupied the 19.36% of the execution time. A further improvement would be to replace the followingCity() with a city-to-position index table and to replace the temporary vectors with fixed-size arrays.

Part4 – Lab1

Please see the directory of lab1 in the zip file.